Universität Stuttgart

Photo credit https://cf.geekdo-images.com/wJfUveg2zpTGn8E1WzEpl__imagepage/img/
_zirp_1cC_/pic3258473.jpg

Jan Theil

Fachpraktikum: Algorithms for OSM data

Agenda

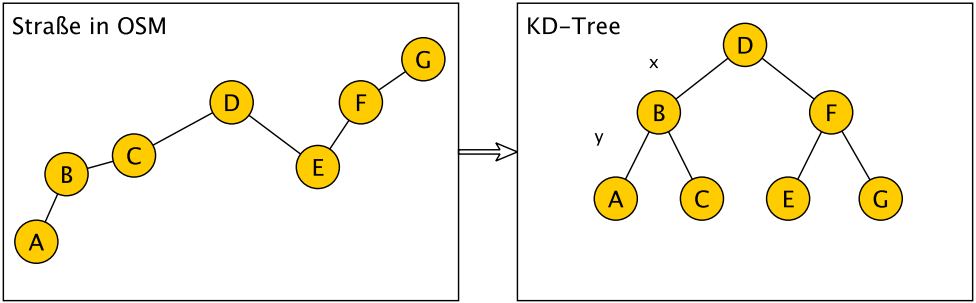
- Object Information
- Schneller PiP-Test mit Bins
- Objects in View
- Substring-Suche (Suffix Array)
- Heuristik
- Closest-to-Query
- Benchmark
- Live Demo

Object Information

- Problem: Es gibt in OSM Gebäude, denen keine Straße zugewiesen wurde
- Ziel: Möglichst passende Straße zuweisen
- Lösung:
  - Balancierter KD-Tree mit allen Straßenpunkten

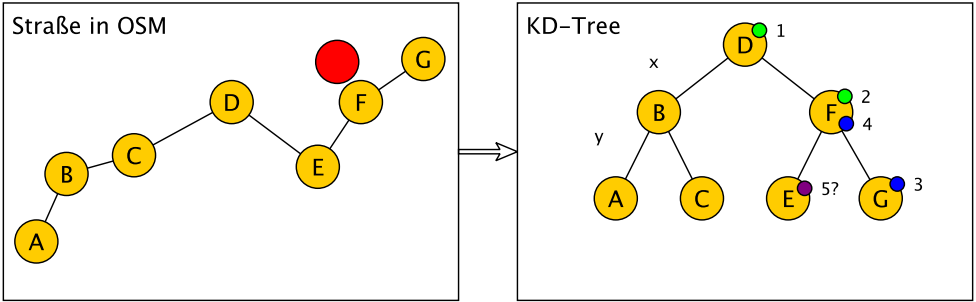
Object Information

- Problem: Es gibt in OSM Gebäude, denen keine Straße zugewiesen wurde
- Ziel: Möglichst passende Straße zuweisen
- Lösung:
  - Balancierter KD-Tree mit allen Straßenpunkten



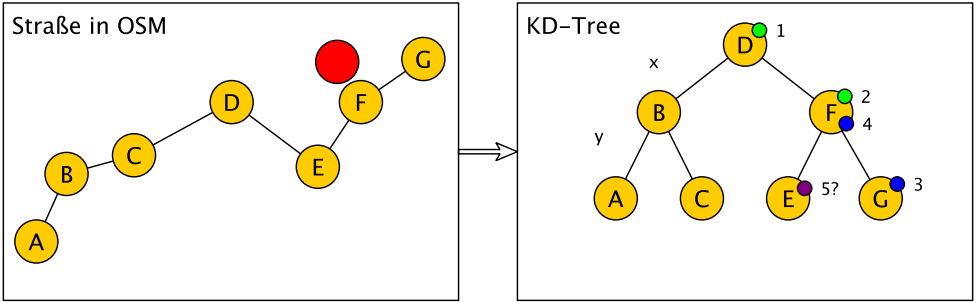
Object Information

- Problem: Es gibt in OSM Gebäude, denen keine Straße zugewiesen wurde
- Ziel: Möglichst passende Straße zuweisen
- Lösung:
  - Balancierter KD-Tree mit allen Straßenpunkten: $\mathcal{O}(n \log(n))$
  - Für alle Gebäude ohne zugewiesene Straße:
    - Finde nächsten Straßenpunkt mit KD-Tree
    - Weise Gebäude diese Straße zu



Object Information

- Problem: Es gibt in OSM Gebäude, denen keine Straße zugewiesen wurde
- Ziel: Möglichst passende Straße zuweisen
- Lösung:
  - Balancierter KD-Tree mit allen Straßenpunkten: $\mathcal{O}(n \log(n))$
  - Für alle Gebäude ohne zugewiesene Straße: Avg. $\mathcal{O}(n \log(n))$, Worst-Case $\mathcal{O}(n^2)$
    - Finde nächsten Straßenpunkt mit KD-Tree
    - Weise Gebäude diese Straße zu

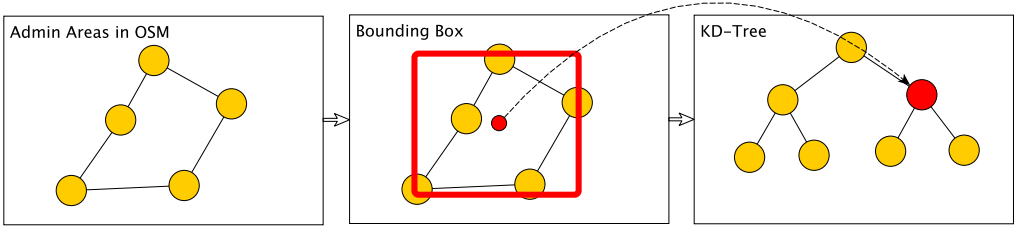


Schneller PiP-Test mit Bins

- Problem: Gebäude in OSM haben keine vollen Adressen
- Ziel: Schnell allen Gebäuden volle Adressen zuweisen
- Lösung (pro Admin. Area Level):
  - Bekannt: Bounding Box / Area, Max. Bounding Box / Level
  - KD-Tree für Bounding Box-Zentren der Areas

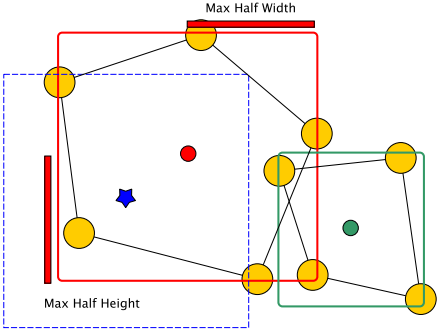
Schneller PiP-Test mit Bins

- Problem: Gebäude in OSM haben keine vollen Adressen
- Ziel: Schnell allen Gebäuden volle Adressen zuweisen
- Lösung (pro Admin. Area Level):
  - Bounding Box / Area, Max. Bounding Box / Level
  - KD-Tree für Bounding Box-Zentren der Areas



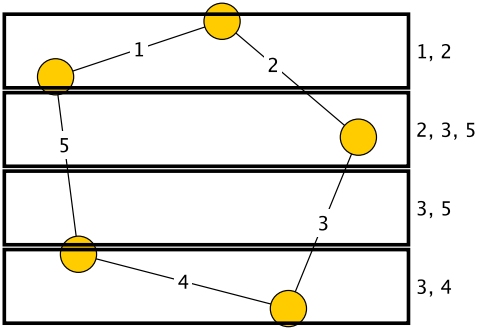
Schneller PiP-Test mit Bins

- Problem: Gebäude in OSM haben keine vollen Adressen
- Ziel: Schnell allen Gebäuden volle Adressen zuweisen
- Lösung (pro Admin. Area Level):
  - Bounding Box / Area, Max. Bounding Box / Level
  - KD-Tree für Bounding Box-Zentren der Areas
  - Filter mithilfe dieser KD-Trees: *KD-Tree.range_search(...)* mit *max_half_width*, *max_half_height*



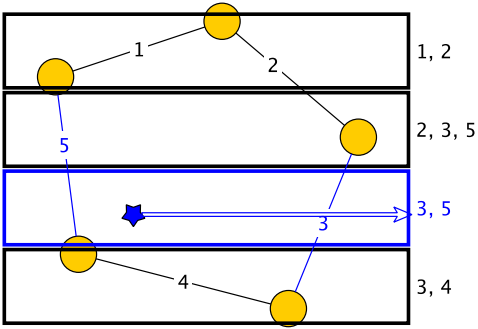
Schneller PiP-Test mit Bins

- Problem: Gebäude in OSM haben keine vollen Adressen
- Ziel: Schnell allen Gebäuden volle Adressen zuweisen
- Lösung (pro Admin. Area Level):
  - Bounding Box / Area, Max. Bounding Box / Level
  - KD-Tree für Bounding Box-Zentren der Areas
  - Filter mithilfe dieser KD-Trees: *KD-Tree.range_search(...)* mit *max_half_width, max_half_height*
  - n (=32) horizontale Spalten über Area mit Kantenreferenzen



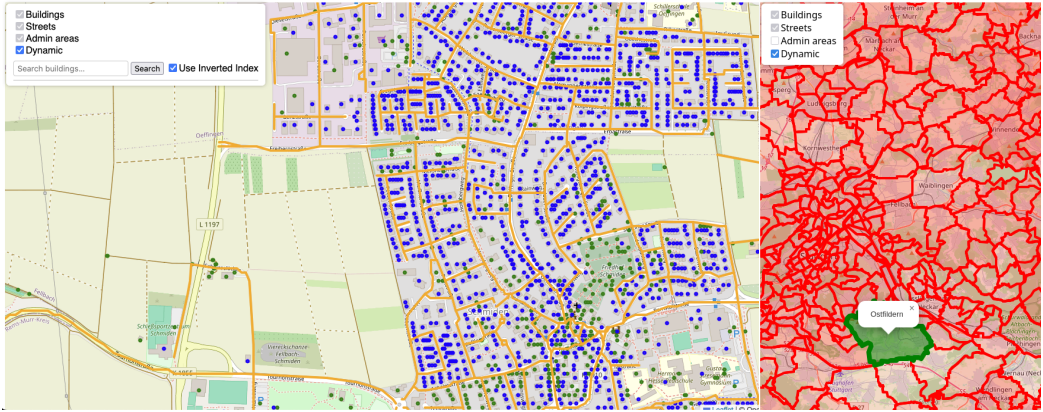
Schneller PiP-Test mit Bins

- Problem: Gebäude in OSM haben keine vollen Adressen
- Ziel: Schnell allen Gebäuden volle Adressen zuweisen
- Lösung (pro Admin. Area Level):
  - Bounding Box / Area, Max. Bounding Box / Level
  - KD-Tree für Bounding Box-Zentren der Areas
  - Filter mithilfe dieser KD-Trees: *KD-Tree.range_search(...)* mit *max_half_width, max_half_height*
  - n (=32) horizontale Spalten über Area mit Kantenreferenzen



Objects in View

- KD-Trees für
  - Gebäude
  - Straßenpunkte
  - Areapunkte
- *KD-Tree.range_search(...)* über Leaflet-Viewbox: $\mathcal{O}(\sqrt{n})$
- APIs für jede KD-Tree-Anfrage, Frontend entscheidet anhand des *zoomLevels*

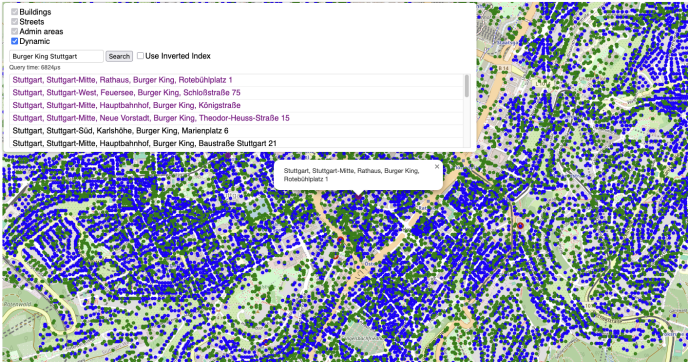


Substring-Suche (Suffix Array)

- Inverted Index bauen
  - Volle Gebäudeadresse umwandeln (ß → ss, ä → ae, Trennen bei Leerzeichen, ...)
  - Dictionary: {<Adresse[...]> : {<Gebäude-ID>, ...}}
- Suffix Array aus *InvertedIndex.keys()*
  - Effiziente Konstuktion mit Rängen: $\mathcal{O}(n \log^2(n))$
    - Sortieren nach $2^0 = 1$ Zeichen
    - Sortieren nach $2^1 = 2$ Zeichen
    - ...
    - Sortieren nach 2^i Zeichen
  - Hinzukommenden Zeichen der nächsten Iteration 2^{i+1} sind Präfix eines anderen Suffixes
  - "Nur noch"Vergleich zweier Zahlen (ranks)
- Query:
  - Querytext umwandeln
  - $\forall q \in query : buildings = search(q) \cap_{galloping} buildings$

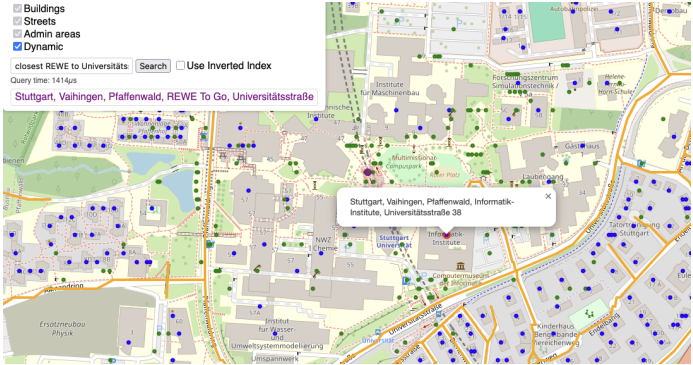
Heuristik

- Rangbasiert
- $rank = 0$
  $if\ (inViewBox())\ rank \mathrel{+}= 200$
  $rank \mathrel{+}= 150 \cdot \dfrac{1}{1 + distanceToCenter()}$
  $rank \mathrel{+}= 180 \cdot queryMatch()$



Closest-to-Query

- Syntax: *Closest* $\langle X \rangle$ *to* $\langle Y \rangle$
- $\widehat{Y} = sort(query(\langle Y \rangle))[0]$
- $\widehat{X} = min_{dist(\cdot,\widehat{Y})}\{query(\langle X \rangle)\}$



Benchmark

- Hardware
  - 16 GB RAM
  - AMD Ryzen 7 3700X 8-Core Processor
    - 16 Threads, 4.4 GHz
- Speicher: Durchschnittlicher maximaler Speicher gemessen über 10 Läufe mit `<sys/resource.h>`
- Zeit: Durchschnittliche Laufzeiten über 100 Läufe
  - AtB = Areas to buildings, SN: Street names to buildings, II: Inverted Index build, SA: Suffix array build
  - 'Stuttgart Bahnhofstrasse 10' (II, SA)
  - 'Closest Rewe to Universitaetsstrasse 38' (SA2)

	Speicher	Zeit (Preprocessing)	Zeit (Querries)
Stuttgart (203 MB) #buildings 1864859 #streets 195645 #admin areas 1465	831 MB	AtB: 4.65s SN: 13.65s II: 4.87s SA: 0.18s Total: 24.39s	II: 0.452ms SA: 4.285ms SA2: 0.148ms
Baden-Württemberg (624 MB) #buildings 5408585 #streets 584820 #admin areas 4415	2377 MB	AtB: 27.10s SN: 77.79s II: 21.81s SA: 0.52s Total: 130.71s	II: 4.563ms SA: 56.715ms SA2: 0.718ms

Tabelle: Becnhmark Ergebnisse

Live Demo